

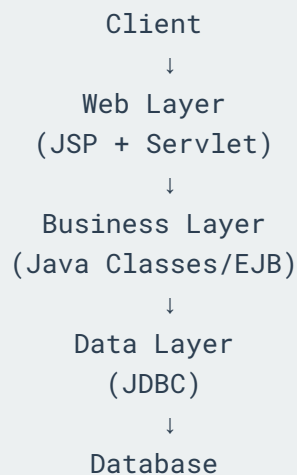
EXAM LAST MIN REVISION UNIT 1-2

UNIT 1 Introduction to Java EE

- Overview of Java EE Architecture
- Difference between Java SE and Java EE
- Role of ... in Web Applications
 - o JDBC: Java Database Connectivity.
 - o JSP: JavaServer Pages.
 - o Servlets: Java Servlets

1. Overview of Java EE Architecture

Easy Memory Trick



Short Form: C W B D DB

Think: Client → Web → Business → Data → Database

Component Comparison

Component	Purpose
Client	Sends request
JSP	User Interface (View)

Servlet	Controller, handles requests
Business Layer	Business Logic
JDBC	Database Connectivity
Database	Stores data

Exam Points

- Java EE is used for enterprise web applications.
- It follows a multi-tier architecture.
- JSP provides presentation layer.
- Servlet acts as controller.
- JDBC connects Java applications to databases.
- Database stores application data.

Quick Revision (2 Marks)

Java EE Architecture consists of Client, Web Layer (JSP and Servlet), Business Layer, Data Layer (JDBC), and Database. JSP displays data, Servlet handles requests, JDBC connects to the database, and the database stores information.

2. Difference Between Java SE and Java EE

Easy Memory Trick

Java SE → Standalone Applications

Java EE → Enterprise Web Applications

Short Form: SE = Standard, EE = Enterprise

Think: Desktop → Java SE, Web App → Java EE

Component Comparison

Feature	Java SE	Java EE
Purpose	Standalone Applications	Enterprise Web Applications
Platform	Desktop	Web/Server
Servlet Support	No	Yes
JSP Support	No	Yes
JDBC	Yes	Yes

Application Server	Not Required	Required
--------------------	--------------	----------

Exam Points

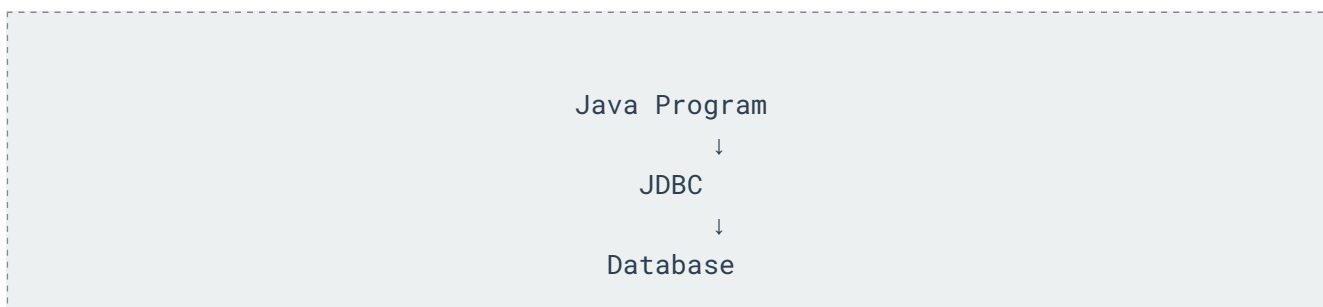
- Java SE is used for desktop applications.
- Java EE extends Java SE.
- Java EE provides Servlets and JSP.
- Java EE supports multi-tier architecture.
- Java EE runs on web/application servers.

Quick Revision (2 Marks)

Java SE is used for standalone desktop applications, whereas Java EE is used for enterprise web applications. Java EE extends Java SE by providing technologies like JSP, Servlets, and application server support.

3. JDBC (Java Database Connectivity)

Easy Memory Trick



Short Form: J D

Think: Java Connects Database

Component Comparison

Component	Purpose
Driver	Communicates with DB
Connection	Establishes DB connection
Statement	Executes SQL
ResultSet	Stores query result
Database	Stores data

Exam Points

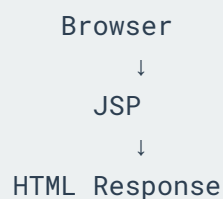
- JDBC stands for Java Database Connectivity.
- JDBC connects Java applications to databases.
- Used for INSERT, UPDATE, DELETE, and SELECT operations.
- Provides Connection, Statement, and ResultSet interfaces.
- Supports transaction management.

Quick Revision (2 Marks)

JDBC is a Java API used to connect Java applications with databases. It provides interfaces such as Connection, Statement, and ResultSet for executing SQL queries and retrieving data.

4. JSP (JavaServer Pages)

Easy Memory Trick



Short Form: J H

Think: Java + HTML = JSP

Component Comparison

Component	Purpose
JSP	Creates dynamic web pages
HTML	Page Design
Java Code	Dynamic Content
Browser	Displays Output

Exam Points

- JSP stands for JavaServer Pages.
- Used to create dynamic web pages.
- Allows embedding Java code inside HTML.
- Simplifies presentation layer development.

- Usually works with Servlets in MVC architecture.

Quick Revision (2 Marks)

JSP is a server-side technology used to create dynamic web pages. It combines HTML with Java code and generates content dynamically before sending it to the browser.

5. Servlets (Java Servlets)

Easy Memory Trick



Short Form: R P R

Think: Receive → Process → Respond

Component Comparison

Component	Purpose
Browser	Sends Request
Servlet	Handles Request
JDBC	Accesses Database
JSP	Displays Result
Response	Sent to Client

Exam Points

- Servlet is a server-side Java program.
- Handles client requests and responses.
- Acts as Controller in MVC architecture.
- Can communicate with databases using JDBC.
- Runs inside a servlet container like Apache Tomcat.

Quick Revision (2 Marks)

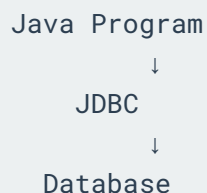
A Servlet is a server-side Java component that processes client requests and generates responses. It acts as the controller in web applications and interacts with JSP and JDBC to build dynamic websites.

JDBC (Java Database Connectivity)

- Introduction to JDBC
- JDBC Drivers and Architecture
- Connecting to Databases
- Executing SQL Queries (SELECT, INSERT, UPDATE, DELETE)

1. Introduction to JDBC

Easy Memory Trick



Short Form: J D

Think: Java Connects Database

Component Comparison

Component	Purpose
Java Application	Sends requests
JDBC API	Connects Java and DB
Database Driver	Communicates with DB
Database	Stores data

Exam Points

- JDBC stands for Java Database Connectivity.
- It is an API used to connect Java applications to databases.
- JDBC allows execution of SQL queries.
- It supports CRUD operations.
- It acts as a bridge between Java and databases.

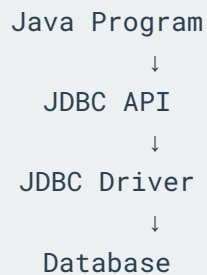
Quick Revision (2 Marks)

JDBC (Java Database Connectivity) is a Java API that enables Java applications to

communicate with databases. It is used to execute SQL queries and perform CRUD operations.

2. JDBC Drivers and Architecture

Easy Memory Trick



Short Form: J A D DB

Think: Java → API → Driver → Database

Component Comparison

Component	Purpose
JDBC API	Provides database interfaces
JDBC Driver	Converts JDBC calls into DB-specific calls
Database	Processes SQL queries
Java Application	Uses JDBC services

Types of JDBC Drivers

Type	Name
Type 1	JDBC-ODBC Bridge Driver
Type 2	Native API Driver
Type 3	Network Protocol Driver
Type 4	Thin Driver (Most Used)

Exam Points

- JDBC architecture consists of Java Application, JDBC API, JDBC Driver, and Database.

- Drivers translate JDBC calls into database-specific commands.
- Four types of JDBC drivers exist.
- Type 4 Driver is the most commonly used driver.

Quick Revision (2 Marks)

JDBC Architecture consists of Java Application, JDBC API, JDBC Driver, and Database. The JDBC Driver translates Java requests into database-specific commands and returns results back to the application.

3. Connecting to Databases

Easy Memory Trick

```

Load Driver
  ↓
Create Connection
  ↓
Use Database
  
```

Short Form: D C U

Think: Driver → Connection → Database

Connection Steps

Step	Purpose
Load Driver	Registers driver
Create Connection	Connects to DB
Create Statement	Executes SQL
Close Connection	Releases resources

Basic Syntax

```

Class.forName("com.mysql.cj.jdbc.Driver");

Connection con =
DriverManager.getConnection(
"jdbc:mysql://localhost:3306/studentdb",
"root",
"password"
);
  
```

Exam Points

- DriverManager is used to establish database connections.
- Connection object represents an active database connection.
- URL, username, and password are required.
- Connections should always be closed after use.

Quick Revision (2 Marks)

Database connection in JDBC is established using `DriverManager.getConnection()`. A Connection object is created using the database URL, username, and password.

4. Executing SQL Queries (SELECT, INSERT, UPDATE, DELETE)

Easy Memory Trick

```
graph TD
    A[Create Statement] --> B[Execute Query]
    B --> C[Get Result]
```

Short Form: S E R

Think: Statement → Execute → Result

SQL Operation Comparison

Operation	SQL Query	JDBC Method
SELECT	SELECT * FROM student	executeQuery()
INSERT	INSERT INTO student VALUES(...)	executeUpdate()
UPDATE	UPDATE student SET ...	executeUpdate()
DELETE	DELETE FROM student WHERE ...	executeUpdate()

Common Syntax

```
// SELECT
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM student");

// INSERT
st.executeUpdate("INSERT INTO student VALUES(1, 'Rahul')");
```

```
// UPDATE
st.executeUpdate("UPDATE student SET name='Amit' WHERE id=1");

// DELETE
st.executeUpdate("DELETE FROM student WHERE id=1");
```

Exam Points

- Statement executes SQL queries.
- `executeQuery()` is used for SELECT.
- `executeUpdate()` is used for INSERT, UPDATE, and DELETE.
- `ResultSet` stores SELECT query results.
- SQL queries are executed through JDBC statements.

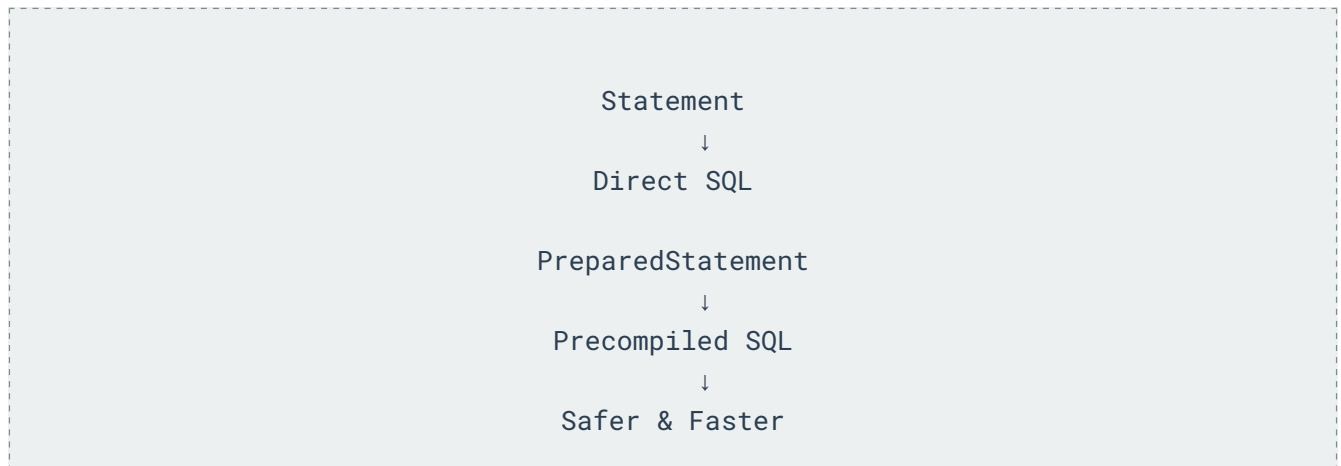
Quick Revision (2 Marks)

JDBC executes SQL queries using Statement or PreparedStatement. SELECT uses `executeQuery()` and returns a `ResultSet`, while INSERT, UPDATE, and DELETE use `executeUpdate()` and return the number of affected rows.

- Use of Statement and PreparedStatement
- Result set and Metadata
- Transaction management
- Batch Processing
- Commit, Rollback, Save-Point

1. Use of Statement and PreparedStatement

Easy Memory Trick



Short Form: S P

Think: Statement = Normal Query, PreparedStatement = Secure Query

Component Comparison

Feature	Statement	PreparedStatement
Query Compilation	Every time	Once
Parameters	Not Supported	Supported (?)
Speed	Slower	Faster
SQL Injection	Vulnerable	Prevents
Reusability	Less	More

Syntax

```

// Statement
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM student");

// PreparedStatement

```

```
PreparedStatement ps = con.prepareStatement("SELECT * FROM student WHERE id=?");
ps.setInt(1,101);
ResultSet rs2 = ps.executeQuery();
```

Exam Points

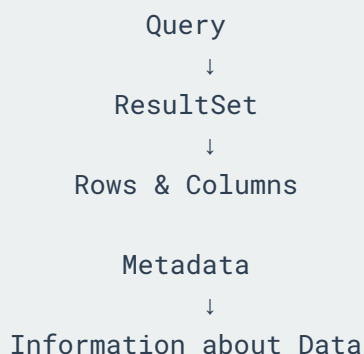
- Statement executes normal SQL queries.
- PreparedStatement executes precompiled SQL queries.
- PreparedStatement supports parameters (?).
- PreparedStatement improves performance.
- PreparedStatement helps prevent SQL Injection.

Quick Revision (2 Marks)

Statement executes SQL queries directly, whereas PreparedStatement executes precompiled parameterized queries. PreparedStatement is faster, reusable, and more secure against SQL injection attacks.

2. ResultSet and Metadata

Easy Memory Trick



Short Form: R M

Think: ResultSet = Actual Data, Metadata = Data about Data

Component Comparison

Component	Purpose
ResultSet	Stores query result
ResultSetMetaData	Information about columns
DatabaseMetaData	Information about database

Syntax

```
// ResultSet
ResultSet rs = st.executeQuery("SELECT * FROM student");
while(rs.next()) {
    System.out.println(rs.getInt("id"));
}

// Metadata
ResultSetMetaData md = rs.getMetaData();
System.out.println(md.getColumnCount());
```

Exam Points

- ResultSet stores records returned by SELECT queries.
- ResultSet cursor points to rows.
- ResultSetMetaData provides column information.
- DatabaseMetaData provides database information.
- Metadata means "data about data".

Quick Revision (2 Marks)

ResultSet stores data returned from a SELECT query. Metadata provides information about the structure of the data, such as column names, types, and counts.

3. Transaction Management

Easy Memory Trick

```
Begin
  ↓
Execute Queries
  ↓
Commit / Rollback
```

Short Form: B E C/R

Think: All Success → Commit, Any Failure → Rollback

Component Comparison

Component	Purpose
Transaction	Group of operations
Commit	Permanently saves changes

Rollback	Cancels changes
Savepoint	Creates checkpoint

Exam Points

- Transaction is a group of SQL operations.
- Ensures data consistency.
- Follows ACID properties.
- Uses Commit and Rollback.
- Prevents partial updates.

Quick Revision (2 Marks)

Transaction management groups multiple database operations into a single unit. If all operations succeed, changes are committed; otherwise, they are rolled back.

4. Batch Processing

Easy Memory Trick

```

Query 1
Query 2
Query 3
  ↓
Batch
  ↓
Execute Together

```

Short Form: A E (Add → Execute)

Think: Many Queries ↓ One Execution

Component Comparison

Method	Purpose
addBatch()	Adds query to batch
executeBatch()	Executes all queries
clearBatch()	Removes batch queries

Syntax

```
Statement st = con.createStatement();
st.addBatch("INSERT INTO student VALUES(1,'A')");
st.addBatch("INSERT INTO student VALUES(2,'B')");
st.addBatch("INSERT INTO student VALUES(3,'C')");
st.executeBatch();
```

Exam Points

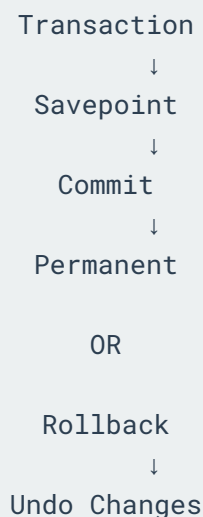
- Batch Processing executes multiple queries together.
- Improves performance.
- Reduces database communication overhead.
- Uses addBatch() and executeBatch().
- Useful for bulk inserts and updates.

Quick Revision (2 Marks)

Batch Processing allows multiple SQL statements to be executed together in a single request, improving performance and reducing database communication time.

5. Commit, Rollback, Savepoint

Easy Memory Trick



Short Form: S C R

Think: Save → Commit, Error → Rollback

Component Comparison

Operation	Purpose
-----------	---------

Commit	Saves changes permanently
Rollback	Cancels changes
Savepoint	Creates checkpoint inside transaction

Syntax

```
con.setAutoCommit(false);
Savepoint sp = con.setSavepoint("P1");
// ... execute queries
con.commit();
// if error: con.rollback(sp);
```

Exam Points

- Commit permanently stores changes.
- Rollback reverses changes.
- Savepoint creates intermediate checkpoints.
- Used in transaction management.
- Auto-commit should be disabled for manual control.

Quick Revision (2 Marks)

Commit permanently saves database changes, Rollback cancels changes, and Savepoint creates a checkpoint within a transaction so that rollback can return to a specific point instead of cancelling the entire transaction.